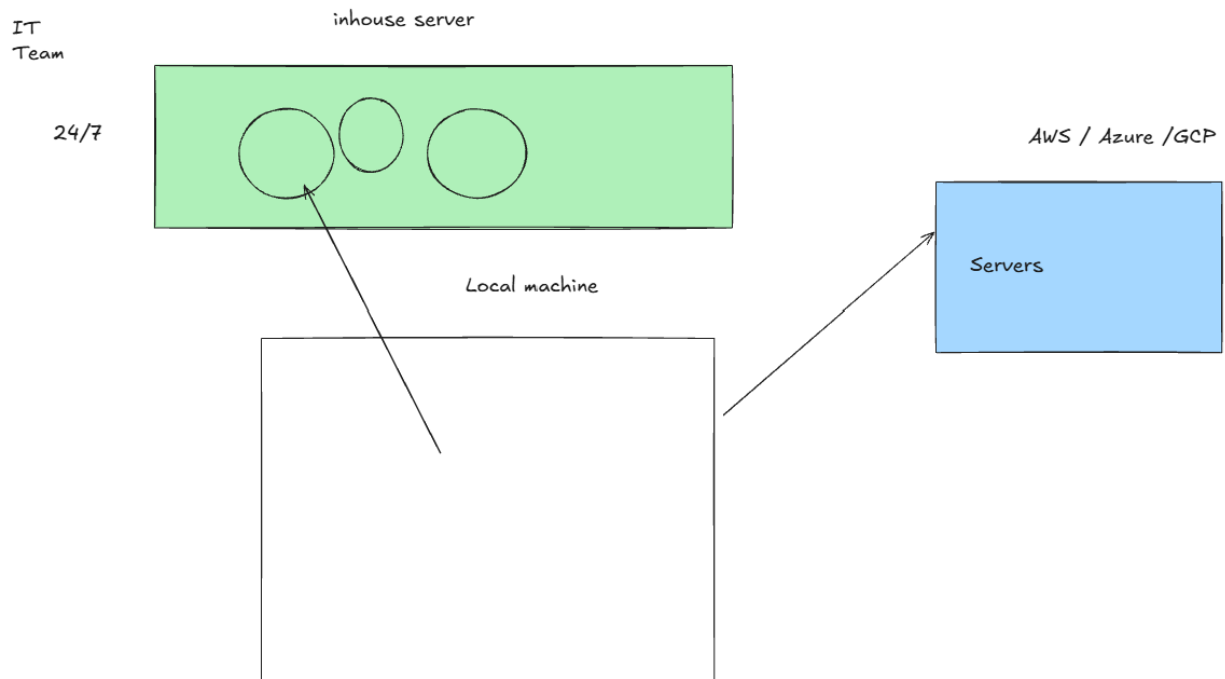
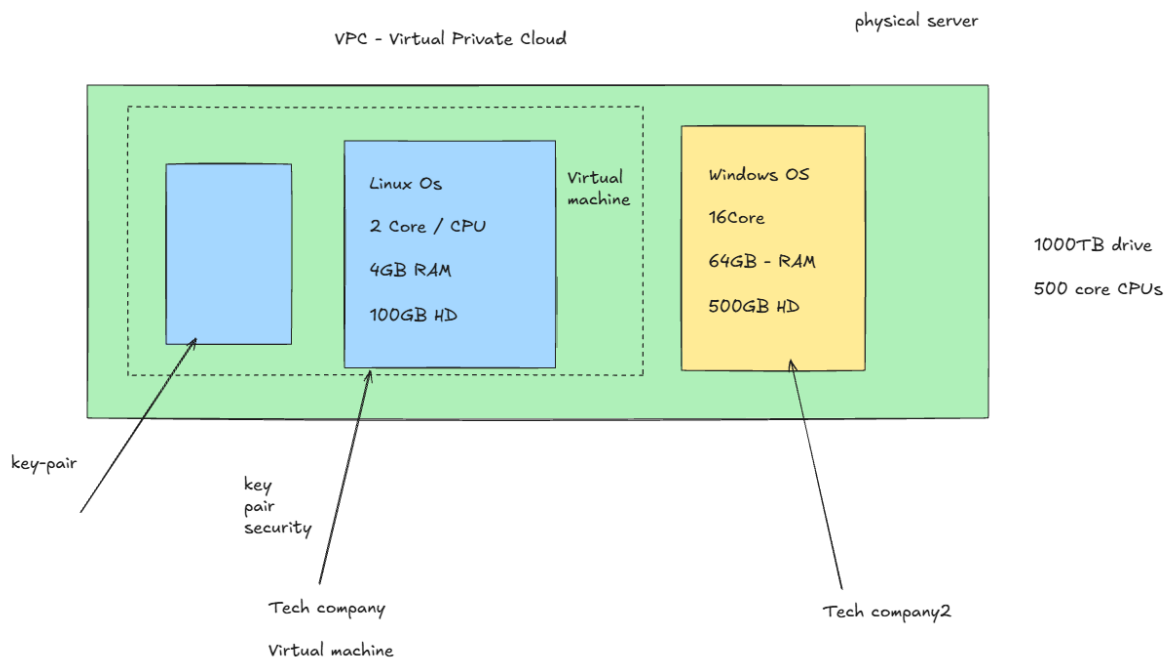


## LIVE NOTES

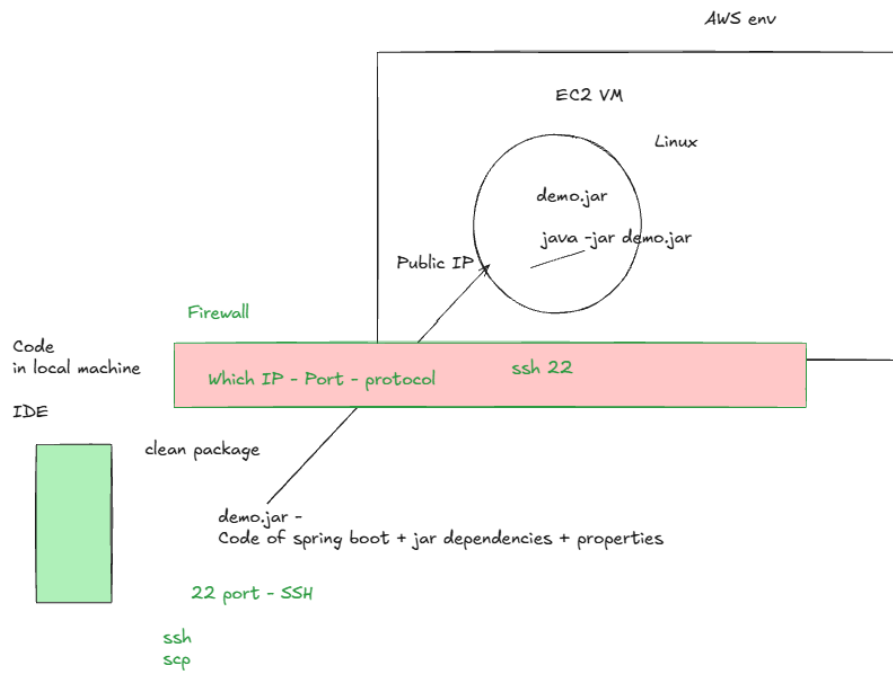
### Application deployment inhouse vs AWS



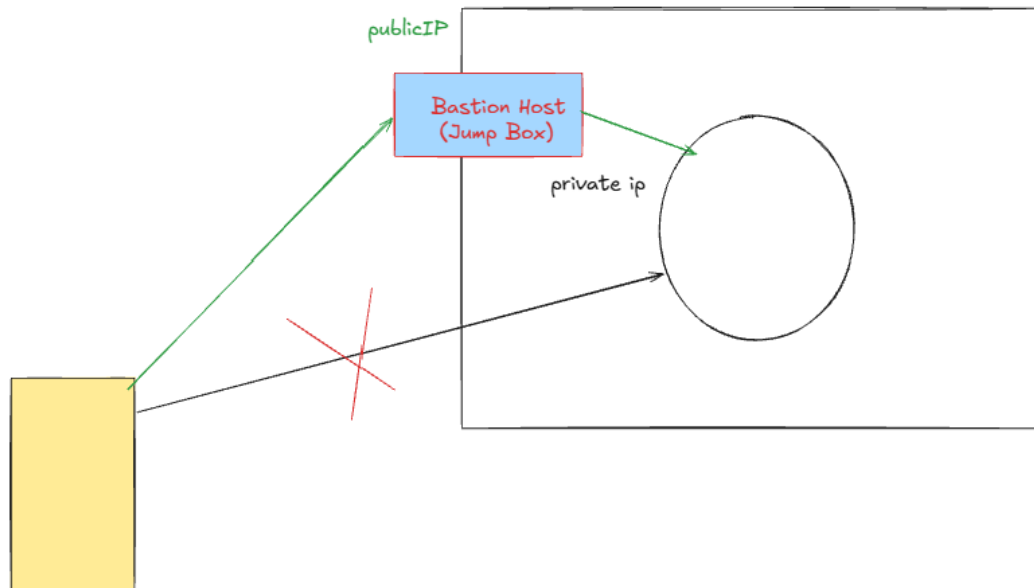
### Virtual machine created on physical machine.



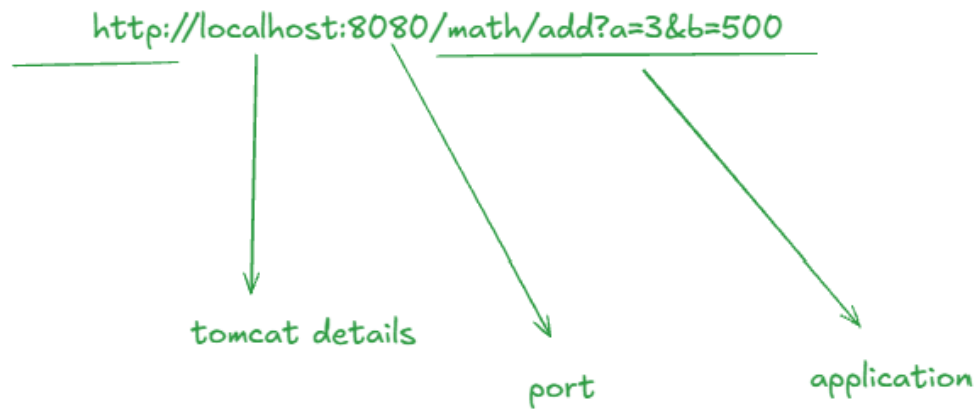
## Deploy your application - in AWS EC2



## Private IP via jumpbox

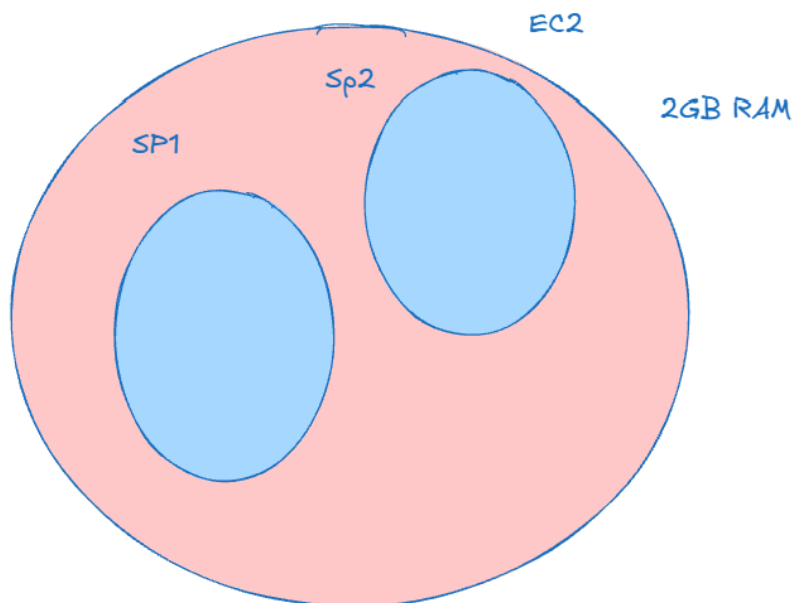


## Making RestAPI call on spring boot app, running in AWS



`http://3.110.131.161:8080/math/add?a=3&b=500`

## Limit the memory allocation for your spring boot application



Working with AWS - deploying our Spring Boot application.

Payment Project - Stripe Payment Integration.

Joining into Week2.

=====

Working with AWS.

Inhouse application deployment.

Cloud application deployment - AWS, Azure, GCP.

Geo Location => Region                      => Availability zones => data centers => Racks  
Asia pacific    => Mumbai / Hyd =>

if you want to create a virtual machine in AWS - you can use AWS EC2 service.

====

Cloud infra - AWS

2mins

====

Devops eng.

3yrs and more.. AWS

dev + infra

0-2

AWS offering multiple services which we can use.

SQL DB

No SQL

Virtual machine

security password

redis cache

asyn broker

...

..

200+ services

AWS offers multiple services, which can support all types of technical projects across the world

AWS Service

EC2 - deploying our spring boot application.

RDS Relation Database - MySQL, oracle...

Secret Manager.

IAMRole

===== Deploy your spring boot app in AWS & test.

1. We need to get EC2 machine in AWS.
2. Install Java in EC2
3. build jar file in your local
4. Copy jar in AWS EC2
5. java -jar
6. Test from local

=====

Software needed to connect to EC2

commands needed to copy jar to destination

install java 17 on AWS

basic Linux commands

===== Mac / Linux

inbuild terminal

=====

===== windows user

Mobaxterm

=====

You will need to open an account in AWS.

Valid CC

free tier credits.

----

Pay as you go modal.

----

whether to setup or not to setup.

As if you are running in AWS< like that we can test in our local machine aswell..

=====

<https://aws.amazon.com/console/>

Setup AWS account & signin.

Mumbai region

Have solutions for problem.

Confident

Willing to experiment

excitement to explore & curiosity should be there..

attitude for building good systems should be there.

I can figure out using internet..

IIITH

1.5hrs

coding late nights.

project coding tasks.. and of new things which they near heard.

3-4weeks

weekdays, weekends, late night..

80+% of jobseekers are not good. They are not ready to work in IT.

1000

800 not good.

top 200

300 people

25-30 fresher

less than 10 people

lower or expectation

only 5 people.

=====

Belong in top 20%, for getting in IT, and building great career.

=====

There are multiple ways to run spring boot application in AWS.

we are discussing the very basic, common, core way.

-----

take EC2 virtual server

install java 17

deploy spring boot app

execute spring boot app

-----

Most common in industry:

EC2  
Elastic Beanstalk  
ECS + Fargate  
EKS (Kubernetes)  
AWS Lambda (specific serverless use cases)  
AWS App Runner (growing in popularity)

====

EC2 Linux

(AMI) - Amazon Machine Image

- ec2-user is default user to connect & work with Linux server.

Using Instance type, CPU & RAM capacity.

2 CPU, 4GB RAM - PROD system

key-pair - RSA security

you will get a PEM file while connecting to AWS you need to use this PEM file  
you can create the key-pair once, and then use the same for multiple EC2 machine.

you can choose network setting, where you want to put this EC2.  
VPC, subnet, AZ

Auto assign public ip = ENABLE.

so our local machine can directly connect to this EC2.

in projects, also jumbox or Bastion host setup is available, where you need to connect to this jumpbox and from there you can connect to private EC2.

jun25ct-key-pair.pem

=====



AMI

ec2-user

Instance Type

CPU, RAM

t2

t3

key-pair

Networking

VPC, Subnet, AZ, PublicIP, Jumpbox(Bastion Host)

Firewall setting

ssh - 22 port

from which IP this operation is allowed.

which IP, which protocol, which port is allowed to connect to this EC2.

for this EC2 config, how many virtual machine you want to create.

ssh => connect to the machine

scp => copy jar file to this machine.

=====

1. Create Ec2 machine

2. Install java 17 on this machine

connect to this ec2

ssh

ssh -i D:\ctdata\aws\jun25ct-key-pair.pem ec2-user@3.110.131.161

MobaXterm

Mac/Linux  
termin

open jdk  
Amazon Corretto java 17

```
=====
1 clear
2 pwd
3 date
4 ll
5 clear
6 history
7 exit
8 history
9 clear
10 java --version
11 dnf search java-17
12 sudo dnf install -y java-17-amazon-corretto
13 java --version
14 exit
15 java --version
16 history
=====
```

-----  
take EC2 virtual server - DONE

install java 17 - DONE  
- connect to machine using ssh  
- use dnf install for java 17 amazon corretto version

deploy spring boot app  
- first test in local if working - DONE  
- build jar  
- copy jar to EC2 using scp command

scp -i D:\ctdata\aws\jun25ct-key-pair.pem ./demo-0.0.1-SNAPSHOT.jar  
ec2-user@3.110.131.161:/home/ec2-user

- ensure write permission is given to your jar on EC2.  
so you can override this jar again with new application jar.

ec2-user

file permission

ec2user: r-x

chmod u+rw \*

Execute spring boot app

java -jar demo-0.0.1-SNAPSHOT.jar

executable jar file

- run this command as a background process.  
so you can work with Linux here itself.  
accidentally ctrl + c will not close your application.

=====

You have to change the way you think , and solve problems.

- think logically,
- understand concept,
- use internet & AI, for coming up with solutions.
  
- if its not working, use internet & AI, to explore the error, and solution
- keep experimenting till problem is solved.

=====

-----

nohup <command> &

nohup <command> > app.log &

nohup java -jar demo-0.0.1-SNAPSHOT.jar > app.log 2>&1 &

vi app.log

tail -f app.log

when new statements are printed in file, it will be visible for you.

ctrl + c

spring boot is still running.

ps -ef

list all processes

application is up.

you want to do testing now.

while setting up the application

firewall rules

anywhere 8080 port - TCP

=====

If you have changes in application, what is your cycle.

1. test and identify what is not working as expected. verify the logs strongly.
2. relate logs to your project code.
3. Test in local & reproduce the problem
4. fix the error in local & test again in local

5. rebuild a new jar file with latest changes - DONE

-----

1. stop previously running service.

Linux: ctrl + r

[ec2-user@ip-172-31-43-228 ~]\$ ps -ef | grep java

ec2-user 27237 27211 0 07:37 pts/2 00:00:13 java -jar demo-0.0.1-SNAPSHOT.jar

```
ec2-user 28142 28027 0 08:05 pts/3 00:00:00 grep --color=auto java
[ec2-user@ip-172-31-43-228 ~]$ kill -9 27237
[ec2-user@ip-172-31-43-228 ~]$
```

take backup

```
27 mkdir bkp
28 ll
29 cp demo-0.0.1-SNAPSHOT.jar ./bkp/
```

deploy the new jar with scp command

start the service using nohup

```
nohup java -jar demo-0.0.1-SNAPSHOT.jar > app.log 2>&1 &
```

you can reserve how much memory needed for your application.

```
nohup java -jar -Xms100M -Xmx150M demo-0.0.1-SNAPSHOT.jar > app.log 2>&1 &
```

----

When you restart EC2 machine, then It will change the IP.

Elastic IP.. retain the same IP across machine restarts.

Stopping our EC2